Catalog Module — Feature Suggestions & Improvements

Commerce Pro · Enterprise E-Commerce Backend Scope:
`com.commerce_pro_backend.catalog` (product, category, brand sub-modules)

Table of Contents

1. Missing Features — Products
2. Missing Features — Categories
3. Missing Features — Brands
4. Missing Features — Search & Discovery
5. Missing Features — Catalog-Wide Operations
6. Improvements — Products
7. Improvements — Categories
8. Improvements — Brands
9. Improvements — Architecture & Cross-Cutting
10. Priority Summary Table

---

1. Missing Features — Products

1.1 Product Cloning / Duplication

What's missing: There is no `POST /products/{id}/duplicate` endpoint.

Why it matters: This is one of the most-used operations in any product catalog admin. A Super Admin creating a new product that is similar to an existing one must re-enter every field manually. Cloning lets them duplicate a product, then change only the name, SKU, and price before publishing.

What to build:

- `POST /products/{id}/duplicate` in `ProductController`
- `ProductService.duplicateProduct(id)` — deep-copies the product, clears `id`, appends `" (Copy)"` to name, generates a new unique SKU (e.g. `ORIGINAL-SKU-COPY-1`), sets `status = "draft"`, clears `salesCount`, `revenue`, `rating`, `reviewCount`, `hasOrders`
- Returns the new `ProductResponseDTO`

1.2 Product Publishing Workflow (Draft → Review → Published)

What's missing: `status` is a free-form string with no enforced state machine. There is no `publishedAt`, `publishedBy`, `archivedAt`, or `reviewRequestedAt` on the `Product` entity.

Why it matters: In an enterprise environment, a product manager creates a draft, a reviewer approves it, and only then does it go live. Without enforcement, anyone with update access can publish immediately, bypassing quality controls.

What to build:

- `ProductStatus` enum: `DRAFT`, `PENDING_REVIEW`, `APPROVED`, `PUBLISHED`, `ARCHIVED`, `REJECTED`
- Add `publishedAt`, `publishedBy`, `archivedAt`, `reviewRequestedAt`, `reviewedBy`, `rejectionReason` fields to `Product`
- `ProductService.submitForReview(id, adminId)` — transitions `DRAFT → PENDING_REVIEW`
- `ProductService.approveProduct(id, adminId)` — transitions `PENDING_REVIEW → APPROVED`
- `ProductService.publishProduct(id, adminId)` — transitions `APPROVED → PUBLISHED`
- `ProductService.rejectProduct(id, adminId, reason)` — transitions `PENDING_REVIEW → REJECTED`
- `ProductService.archiveProduct(id, adminId)` — transitions any state → `ARCHIVED` (soft archive, not hard delete)
- Guard all transitions with illegal-state exceptions

1.3 Product Scheduling (Publish / Unpublish at Future Date)

What's missing: No `scheduledPublishAt` or `scheduledUnpublishAt` fields on `Product`. No scheduler to action them.

Why it matters: Super Admins managing seasonal or promotional products need to say "publish this product on Dec 1st and take it down on Jan 5th" without being online at midnight to do it manually.

What to build:

- Add `scheduledPublishAt` (`LocalDateTime`) and `scheduledUnpublishAt` (`LocalDateTime`) to `Product`
- `ProductSchedulerService` with `@Scheduled(cron = "0 * * * * *")` (runs every minute)
  - Finds products where `scheduledPublishAt <= now AND status = APPROVED` → calls `publishProduct()`
  - Finds products where `scheduledUnpublishAt <= now AND status = PUBLISHED` → calls `archiveProduct()`
- `PATCH /products/{id}/schedule` endpoint accepting `scheduledPublishAt` and `scheduledUnpublishAt`

1.4 Product Versioning / Change History

What's missing: No audit trail for product edits. `Product` has `createdAt` / `updatedAt` but no history of what changed, when, or by whom.

Why it matters: A Super Admin cannot see what a product's price was 3 weeks ago, who changed the description, or roll back after an incorrect bulk update. This is a compliance requirement in many enterprise environments.

What to build:

- `ProductChangeLog` entity: `id` , `productId` , `changedBy` , `changedAt` , `fieldName` , `oldValue` , `newValue` , `changeType` (CREATED / UPDATED / STATUS_CHANGED / PRICE_CHANGED / STOCK_CHANGED)
- `ProductChangeLogRepository` and `ProductChangeLogService`
- In `ProductService.updateProduct()` and `patchProduct()` , compare old vs new values field-by-field and write change log entries

- `GET /products/{id}/history` — paginated change log for a product
- `POST /products/{id}/history/{logId}/revert` — restore a specific field to its previous value

1.5 Multi-Image Management

What's missing: `Product` has `gallery` (List<String>), `imageUrl`, and `featuredImage`, but no API to add, remove, or reorder individual gallery images. A Super Admin must PUT the entire product just to delete one photo.

Why it matters: Image management is a frequent, standalone operation. Replacing the whole product to manage one image is unsafe and inefficient.

What to build:

- `POST /products/{id}/images` — add one or more images to gallery (multipart or URL list)
- `DELETE /products/{id}/images/{index}` — remove a specific gallery image by position
- `PATCH /products/{id}/images/reorder` — accepts ordered list of image URLs, updates gallery order
- `PUT /products/{id}/images/featured` — set the featured image from the gallery
- Wire to `FileStorageService.storeProductImage()` which already exists in common module

1.6 Variant-Level Pricing, SKU, and Stock

What's missing: `ProductVariant` only has `name` (String) and `options` (List<String>). No price, no SKU, no stock, no image per variant.

Why it matters: You cannot model a blue shirt at $29 and a red shirt at $35 — there is one price on the parent product covering all variants. This makes the variant system essentially decorative for any real catalog.

What to build:

- Add to `ProductVariant`: `sku` (unique), `price` (BigDecimal, nullable — falls back to parent), `compareAtPrice`, `cost`, `stock`, `weight`, `imageUrl`, `barcode`, `isAvailable` (Boolean)

- `ProductVariantRepository` extending `JpaRepository<ProductVariant, String>`
- `ProductVariantService`: `createVariant()`, `updateVariant()`, `deleteVariant()`, `adjustVariantStock()`
- `POST /products/{id}/variants` — add a variant
- `PUT /products/{id}/variants/{variantId}` — update a variant
- `DELETE /products/{id}/variants/{variantId}` — remove a variant
- `POST /products/{id}/variants/{variantId}/stock` — adjust variant stock
- `ProductService.updateStock()` should aggregate variant stock to compute parent-level `stock` field

---

1.7 Product Import (CSV / Excel)

What's missing: No bulk import endpoint. `ProductController` has `bulk-delete` and `bulk-status` but no `POST /products/import`.

Why it matters: A Super Admin managing thousands of SKUs cannot create products one by one. CSV/Excel import is a day-one expectation for any enterprise catalog management system.

What to build:

- `POST /products/import` — accepts `multipart/form-data` with a CSV or Excel file
- `ProductImportService.importFromFile(MultipartFile)`:
  - Parse rows (Apache Commons CSV already available as transitive dependency)
  - Validate each row independently (SKU uniqueness, required fields, value ranges)
  - Insert or upsert based on SKU (configurable via `mode=CREATE_ONLY | UPDATE_ONLY | UPSERT`)
  - Collect per-row errors without stopping the whole batch
- Returns `ProductImportResult`: `totalRows`, `succeeded`, `failed`, `skipped`, `errors` (list of row number + message)
- `GET /products/import/template` — download a blank CSV template with correct headers

1.8 Product Export (CSV / Excel)

What's missing: `ProductService.getAllProducts()` returns all products as a list, but no endpoint produces a downloadable file.

Why it matters: Super Admins need to export the catalog for offline analysis, sharing with suppliers, data backup, or migration. This is a standard catalog management feature.

What to build:

- `GET /products/export?format=csv|xlsx&status=active&category=electronics` — streams a file download
- `ProductExportService.exportToCsv(ProductFilterDTO)` — applies the same `ProductSpecification` filters, writes to `ByteArrayOutputStream`, returns as `InputStreamResource`
- Supports same filter params as `GET /products` so exports can be scoped
- Includes all fields: id, sku, name, category, brand, price, compareAtPrice, cost, stock, status, variants, tags, seoTitle, etc.

1.9 Related Products / Cross-sell / Up-sell

What's missing: No `relatedProducts`, `crossSellProducts`, or `upSellProducts` relationship on `Product`.

Why it matters: "Customers also bought" and "You might also like" are foundational e-commerce features. A Super Admin needs to curate these relationships to drive basket size.

What to build:

- `ProductRelation` entity: `id`, `sourceProductId`, `targetProductId`, `relationType` (RELATED / CROSS_SELL / UP_SELL), `sortOrder`, `createdAt`, `createdBy`
- `ProductRelationRepository`
- `ProductRelationService`: `addRelation()`, `removeRelation()`, `getRelatedProducts()`, `getCrossSellProducts()`, `getUpSellProducts()`, `reorderRelations()`
- `GET /products/{id}/related`
- `POST /products/{id}/related` — add related product IDs with type
- `DELETE /products/{id}/related/{targetId}`

- `PATCH /products/{id}/related/reorder`

1.10 Product Archiving (Replace Hard Delete)

What's missing: `ProductService.deleteProduct()` calls `productRepository.delete(product)` — a **hard delete**. There is no `archivedAt` field, no `isArchived` flag. A product deleted is gone forever.

Why it matters: Any product that has ever been ordered or viewed should never be hard-deleted. Sales history, reports, and order line items all reference product data. Hard-deleting breaks historical records.

What to build:

- Add `isArchived` (Boolean), `archivedAt` (LocalDateTime), `archivedBy` (String) to `Product`
- `ProductService.archiveProduct(id, adminId)` — sets `isArchived = true`, `status = ARCHIVED`, records who/when
- `ProductService.restoreProduct(id, adminId)` — sets `isArchived = false`, resets to `DRAFT`
- Change `deleteProduct()` to call `archiveProduct()` by default
- Add `?forceDelete=true` param on `DELETE /products/{id}` gated behind a high-privilege check — only for products with no orders and no inventory history
- All `getProducts()` and `searchProducts()` queries should add `isArchived = false` to their default specification

---

1.11 Product Comparison

What's missing: No `GET /products/compare?ids=a,b,c` endpoint.

Why it matters: A Super Admin reviewing catalog quality or configuring a customer-facing comparison feature needs to compare specs, prices, and attributes across multiple products side-by-side in one API call.

What to build:

- `GET /products/compare?ids=id1,id2,id3` (max 5 IDs)
- `ProductService.compareProducts(List<String> ids)` — fetches all, returns a

`ProductComparisonDTO` containing: the list of full `ProductResponseDTO` objects plus a `differenceMap` highlighting fields where values differ across the set

- Validates all IDs exist and belongs to the same category (optional warning, not blocking)

1.12 Bulk Price Update

What's missing: No `POST /products/bulk-price-update` accepting a category/brand filter plus a percentage or fixed adjustment.

Why it matters: A Super Admin cannot raise all Electronics prices by 5% before a product launch without updating each product individually. Bulk pricing is a fundamental admin operation.

What to build:

- `POST /products/bulk-price-update` accepting: filter (`categoryId` , `brandId` , `tag` , or explicit `ids` list), `adjustmentType` (PERCENTAGE / FIXED), `adjustmentValue` , `adjustmentDirection` (INCREASE / DECREASE), `applyToComparePrice` (Boolean)
- `ProductService.bulkUpdatePrice(BulkPriceUpdateRequest)` — applies adjustment, recalculates, saves
- Returns `BulkOperationResult` : `totalMatched` , `updated` , `failed` , preview of first 5 affected products with old/new price
- Optional: `dryRun=true` param that returns what would change without saving

---

2. Missing Features — Categories

2.1 Category → Product Assignment & Count

What's missing: `Product.categoryId` stores a string but has no FK to `Category` . There is no assignment endpoint and no `productCount` field on `Category` .

Why it matters: A Super Admin managing categories cannot see how many products are in each node, and cannot assign products to categories from within the category management section.

What to build:

- Add `productCount` (Integer) field to `Category` entity, updated by a `@Query` or trigger when products are assigned

- `GET /categories/{id}/products` — paginated list of products belonging to this category
- `POST /categories/{id}/products/bulk-assign` — assigns a list of product IDs to this category
- `DELETE /categories/{id}/products/{productId}` — removes a product from the category
- Enforce `categoryId` FK on `Product` as a proper `@ManyToOne` to `Category`

2.2 Category Attribute Templates

What's missing: No way to define which attributes are required or available for products within a category. The current attribute system is entirely free-form per product.

Why it matters: An Electronics category should require "Voltage" and "Warranty Period" while Clothing requires "Size" and "Material." Without templates, catalog data quality is inconsistent — every product has different attribute names for the same concept.

What to build:

- `CategoryAttributeTemplate` entity: `id`, `categoryId`, `attributeName`, `attributeType` (TEXT / NUMBER / BOOLEAN / SELECT), `allowedValues` (List<String> for SELECT type), `isRequired` (Boolean), `displayOrder`, `unit` (e.g. "kg", "cm")
- `CategoryAttributeTemplateService`: `getTemplateForCategory(categoryId)` (walks up hierarchy and merges templates), `createTemplate()`, `updateTemplate()`, `deleteTemplate()`
- `GET /categories/{id}/attribute-templates`
- `POST /categories/{id}/attribute-templates`
- `PUT /categories/{id}/attribute-templates/{templateId}`
- `DELETE /categories/{id}/attribute-templates/{templateId}`
- `ProductService.validateProductAttributes()` — validates product attributes against its category's template before saving

---

2.3 Bulk Product Move Between Categories

What's missing: No way to move multiple products from one category to another in a single

operation.

Why it matters: When a Super Admin restructures the category tree (e.g. splitting "Electronics" into "Phones" and "Laptops"), they need to reassign hundreds of products efficiently.

What to build:

- `POST /categories/{id}/products/bulk-move` accepting `targetCategoryId` and `productIds` list
- `CategoryService.bulkMoveProducts(sourceCategoryId, targetCategoryId, productIds)` — validates target exists, updates `Product.categoryId` for all matching products, updates `productCount` on both source and target categories
- Returns `BulkOperationResult` with per-product success/failure

2.4 Category-Level Pricing Rules

What's missing: No way to define default tax class, minimum margin, or pricing constraints at the category level.

Why it matters: Enterprise catalogs enforce pricing rules by category — Electronics might have a 15% minimum margin, Groceries might be zero-rated for tax. Without this, pricing logic must be manually applied to every individual product.

What to build:

- `CategoryPricingRule` entity: `id`, `categoryId`, `taxClass` (STANDARD / REDUCED / ZERO / EXEMPT), `minimumMarginPercent`, `defaultMarkupPercent`, `currencyCode`, `inheritsFromParent` (Boolean)
- `GET /categories/{id}/pricing-rule`
- `PUT /categories/{id}/pricing-rule`
- `ProductService.validateProductPricing()` — warns (or blocks) if a product's margin falls below its category's `minimumMarginPercent`

---

3. Missing Features — Brands

3.1 Brand Product Listing

What's missing: No `GET /brands/{id}/products` endpoint. A Super Admin clicking a brand in the admin panel cannot see all products under that brand.

Why it matters: Brand management is incomplete without being able to view and filter the products associated with a brand directly.

What to build:

- `GET /brands/{id}/products` — paginated, filtered product list for a given brand ID (reuses `ProductSpecification` with brand filter)
- `GET /brands/{id}/products/stats` — count by status, total stock value, average price, average rating for the brand

---

3.2 Brand Analytics

What's missing: `Brand` only has `productCount`. No revenue, no sales count, no top product. No analytics endpoint.

Why it matters: A Super Admin needs to know which brand generates the most revenue, which has the most out-of-stock products, and which brand's products perform best — to inform purchasing and promotional decisions.

What to build:

- `BrandAnalyticsDTO`: `brandId`, `brandName`, `totalProducts`, `activeProducts`, `outOfStockCount`, `lowStockCount`, `totalRevenue`, `totalSalesCount`, `avgRating`, `topProductId`, `topProductName`
- `BrandService.getBrandAnalytics(id)` — aggregates from `ProductRepository` using JPQL
- `GET /brands/{id}/analytics`
- `GET /brands/analytics/summary` — ranked list of all brands by revenue or sales count

---

3.3 Brand Merge / Consolidation

What's missing: No `POST /brands/{id}/merge/{targetId}` endpoint.

Why it matters: Duplicate brands ("Nike" and "Nike Inc.") are a common data quality problem. Without a merge operation, a Super Admin must update each product manually.

What to build:

- `POST /brands/{id}/merge` accepting `{ "targetBrandId": "..." }` in body
- `BrandService.mergeBrands(sourceBrandId, targetBrandId)`:
  - Reassigns all products from source brand to target brand
  - Updates `targetBrand.productCount` to combined total
  - Soft-deletes the source brand
  - Writes an audit log entry
- Returns merged brand DTO plus count of reassigned products

3.4 Brand SEO Fields

What's missing: `Brand` has `name`, `slug`, `description`, `logo`, `website` but no SEO metadata — unlike `Category` which has `seoTitle`, `seoDescription`, `metaKeywords`.

Why it matters: Brand pages on storefronts need their own SEO metadata for search engine indexing.

What to build:

- Add `seoTitle` (String, max 70), `seoDescription` (String, max 160), `metaKeywords` (String, max 500), `canonicalUrl` (String) to `Brand` entity
- Include in `BrandDto.Request` and `BrandDto.Response`

---

4. Missing Features — Search & Discovery

4.1 Search Synonym Management

What's missing: No synonym or alias management. If a customer searches "sofa" but products say "couch," they find nothing.

Why it matters: Search relevance is critical for e-commerce. Synonyms directly affect conversion rates. A Super Admin should be able to define term mappings without a code deployment.

What to build:

- `SearchSynonym` entity: `id`, `term`, `synonyms` (List<String>), `isActive`, `createdAt`, `createdBy`

- `SearchSynonymService` : `getSynonymsForTerm(query)` — returns all synonym terms for a given search query
- `ProductRepository.search()` — extend to expand the query with synonym terms before building LIKE predicates
- `GET /catalog/search/synonyms` — list all synonym groups
- `POST /catalog/search/synonyms` — add a synonym group
- `PUT /catalog/search/synonyms/{id}` — update
- `DELETE /catalog/search/synonyms/{id}` — delete

4.2 Product Ranking / Boost Rules

What's missing: Search results order is fixed by `Pageable` sort. No pinning, boosting, or bury rules. No `boostScore` on `Product` .

Why it matters: A Super Admin running a promotion needs to pin certain products to the top of category or search pages without changing their overall sort field.

What to build:

- Add `boostScore` (Integer, default 0) to `Product` — higher values surface the product earlier in results
- `ProductSpecification` — add `ORDER BY boostScore DESC, <user-selected sort>` as default ordering
- `SearchBoostRule` entity: `id` , `triggerType` (KEYWORD / CATEGORY / TAG), `triggerValue` , `targetProductId` , `boostValue` , `validFrom` , `validUntil`
- `PATCH /products/{id}/boost` — set boost score for a product
- `POST /catalog/search/boost-rules` — create a boost rule
- `GET /catalog/search/boost-rules` — list active boost rules

4.3 Faceted Search Configuration

What's missing: Filter facets (status, category, brand, price range, rating, featured, stock) are hardcoded in `ProductSpecification` . A Super Admin cannot configure which facets are shown or in what order.

Why it matters: Different storefronts and product types need different facet configurations. Electronics might show "Brand" and "Price" prominently while Fashion shows "Size" and "Color."

What to build:

- `FacetConfiguration` entity: `id`, `context` (GLOBAL / CATEGORY-specific), `contextId`, `facetType` (BRAND / CATEGORY / PRICE_RANGE / RATING / TAG / ATTRIBUTE), `label`, `isVisible`, `displayOrder`, `isCollapsible`, `maxOptions`
- `GET /catalog/search/facets` — returns active facet config for given context
- `PUT /catalog/search/facets` — update facet configuration
- `GET /products/facets?category=electronics` — returns dynamic facet values (e.g. all brand names in that category with product counts) based on the facet config

---

5. Missing Features — Catalog-Wide Operations

5.1 Catalog Audit Log

What's missing: No audit trail for any catalog operation. `ProductService`, `CategoryService`, and `BrandService` never call `AuditService`. A Super Admin cannot answer "who changed that price" or "who deleted that category."

Why it matters: This is the most critical missing feature for an enterprise system. Accountability and traceability are non-negotiable at enterprise scale.

What to build:

- `CatalogAuditLog` entity: `id`, `entityType` (PRODUCT / CATEGORY / BRAND), `entityId`, `entityName`, `action` (CREATED / UPDATED / DELETED / ARCHIVED / PUBLISHED / PRICE_CHANGED / STATUS_CHANGED / STOCK_CHANGED), `actorId`, `actorUsername`, `fieldChanges` (JSON map of field → {old, new}), `ipAddress`, `timestamp`
- `CatalogAuditService.log(entityType, entityId, action, actorId, fieldChanges)` — called from all write operations in all three services
- `GET /catalog/audit` — paginated, filterable by entity type, entity ID, actor, action, date range
- `GET /catalog/audit/products/{id}` — full history for one product

- `GET /catalog/audit/categories/{id}` — full history for one category
- `GET /catalog/audit/brands/{id}` — full history for one brand

5.2 Catalog Health Dashboard

What's missing: `GET /products/stats` returns totals but no quality metrics. There is no endpoint reporting products missing images, descriptions, SEO fields, or category assignment.

Why it matters: Catalog data quality directly affects conversion rates and SEO. A Super Admin needs actionable insight into which products need attention, not just raw counts.

What to build:

- `CatalogHealthService.getHealthReport()` returns `CatalogHealthDTO`:
  - `missingImage` — products with null `imageUrl`
  - `missingDescription` — products with null or empty `description`
  - `missingSeoTitle` — products missing `seoTitle`
  - `missingSeoDescription` — products missing `seoDescription`
  - `missingCategory` — products with null `categoryId`
  - `noSalesIn90Days` — products with `salesCount = 0` and `createdAt < 90 days ago`
  - `duplicateUrlHandles` — products sharing the same `urlHandle`
  - `zeroStockActive` — active products with `stock = 0` and `allowBackorders = false`
  - `overallHealthScore` — 0–100 weighted score
- `GET /catalog/health` — returns the full health report
- `GET /catalog/health/issues` — paginated list of products with specific issues, filterable by issue type

---

5.3 SEO Audit

What's missing: `Product` has `seoTitle`, `seoDescription`, `imageAlt`, `urlHandle`. `Category` has `seoTitle`, `seoDescription`, `metaKeywords`. But no endpoint surfaces SEO

gaps or duplicates.

Why it matters: A Super Admin responsible for organic traffic needs to audit and fix SEO metadata across thousands of products efficiently.

What to build:

- `GET /catalog/seo/audit` — returns:
  - Products missing `seoTitle` (list + count)
  - Products missing `seoDescription`
  - Products with `seoTitle` over 70 characters
  - Products with `seoDescription` over 160 characters
  - Products with duplicate `urlHandle` values (grouped)
  - Products with missing `imageAlt`
  - Categories missing `seoTitle` or `seoDescription`
- `GET /catalog/seo/duplicates` — products or categories sharing the same `urlHandle` or `seoTitle`
- `POST /catalog/seo/auto-generate` — bulk-generates `seoTitle` and `seoDescription` from product `name` + `shortDescription` for all products where they are currently null

---

6. Improvements — Products

6.1 Replace String Status with Enum

Current state: `Product.status`, `Product.visibility`, `Product.stockStatus` are all free-form `String` fields.

Problem: Nothing prevents saving `status = "actve"` (typo). There are no compile-time guarantees, no validation, and no IDE support.

Improvement:

- Create `ProductStatus` enum: `DRAFT`, `PENDING_REVIEW`, `PUBLISHED`, `ARCHIVED`, `DISCONTINUED`
- Create `ProductVisibility` enum: `VISIBLE`, `HIDDEN`, `PASSWORD_PROTECTED`, `SCHEDULED`

- Create `StockStatus` enum: `IN_STOCK`, `LOW_STOCK`, `OUT_OF_STOCK`, `BACKORDER`, `PRE_ORDER`
- Replace String fields with enums on `Product` entity
- Use `@Enumerated(EnumType.STRING)` for DB storage
- Update `ProductFilterDTO`, `ProductSpecification`, `ProductMapper` accordingly

6.2 Fix Bulk Operations to Return Results

Current state: `POST /products/bulk-delete` and `POST /products/bulk-status` loop through IDs calling individual methods. If one fails, the whole operation throws with no indication of which IDs succeeded.

Problem: A Super Admin bulk-deleting 50 products has no idea what happened if one of them throws.

Improvement:

- Wrap each individual operation in a try-catch
- Accumulate per-item results: `{ id, success, errorMessage }`
- Return `BulkOperationResult`: `totalRequested`, `succeeded`, `failed`, `results` (per-item detail)
- Never let a single failure abort the batch

6.3 `getAllProducts()` Should Not Load Everything

Current state: `ProductService.getAllProducts()` calls `productRepository.findAll()` with no pagination — fetches every product in the database into memory.

Problem: With 50,000 products this will cause an OOM or a 30-second timeout. The method is intended for exports but is not streamed.

Improvement:

- Remove `getAllProducts()` from `ProductService` or restrict it to small datasets with a max-count guard
- Replace export use case with `ProductExportService` using a scrollable `JpaRepository` query or `Stream<Product>` with `@Transactional(readOnly = true)` and `@QueryHints`

for streaming

- The `GET /products` endpoint already supports pagination and should be the primary list API

6.4 `ProductRepository.search()` Uses Full-Table LIKE Scan

Current state: `ProductRepository.search()` uses `LIKE '%query%'` on four columns — `name`, `sku`, `description`, `brand`. Leading wildcard (`%query%`) means the database cannot use any index.

Problem: On large catalogs this becomes a full-table scan on every keystroke in the search box.

Improvement:

- Short-term: Add a `@FullTextField` virtual column or use H2's `FULLTEXT` / MySQL's `MATCH … AGAINST` via a native query
- Medium-term: Add `@Column(columnDefinition = "TEXT")` with a generated `search_vector` column updated by `@PrePersist` / `@PreUpdate`, indexed with a full-text index
- Long-term: Integrate Spring Data Elasticsearch or Apache Solr for production-grade full-text search

---

6.5 `patchProduct()` Uses Unsafe Reflection-Style Map

Current state: `patchProduct(id, Map<String, Object> updates)` uses a switch statement on field name strings. Unknown fields are silently ignored. There is no validation or type safety.

Problem: A caller can send `{ "price": "not-a-number" }` and get a `ClassCastException` at runtime. The switch only handles 6 fields — all others are silently dropped.

Improvement:

- Replace `Map<String, Object>` with a dedicated `ProductPatchDTO` using `@JsonInclude(NON_NULL)` and individual nullable fields
- All fields present in the DTO are patched; null fields are left unchanged
- Validation annotations apply normally to the DTO fields

- Remove the manual switch statement

6.6 `deleteProduct()` Has No Cascade Consideration

Current state: `productRepository.delete(product)` hard-deletes the product. No consideration for cascade effects on `ProductVariant`, `ProductAttribute`, gallery images, or inventory records.

Problem: Deleting a product orphans related data in other tables and potentially leaves `Inventory` records pointing at a non-existent product.

Improvement:

- Before deleting, check for related `Inventory` records (from the inventory module) and reject or clean up
- Delete associated `FileStorageService`-managed images from disk
- Cascade delete `ProductVariant` and `ProductAttribute` via JPA `CascadeType.ALL` (already configured) — verify orphanRemoval is correct
- Log the deletion with the catalog audit service

6.7 Image URLs Stored as Base64 in DB

Current state: `imageUrl`, `gallery`, and `featuredImage` columns use `columnDefinition = "TEXT"` because they may store base64 data URLs. A single base64 image can be 500KB+ stored directly in the database row.

Problem: Storing binary image data as base64 in the database bloats table size, slows queries, and prevents CDN-based delivery.

Improvement:

- Enforce that only URLs (not base64 data) are stored in image columns
- `ProductService.createProduct()` and `updateProduct()` should reject base64-encoded image values with a 400 error and a message directing the caller to use `POST /products/{id}/images` instead
- `FileStorageService` already exists — require all images to be uploaded through it and store only the returned URL

---

7. Improvements — Categories

7.1 `deleteCategory()` Does Not Check Products

Current state: `CategoryService.deleteCategory()` checks for subcategories but does **not** check whether any products are assigned to the category being deleted.

Problem: Deleting a category that has 200 products leaves those products with a dangling `category` / `categoryId` reference pointing at a soft-deleted entity.

Improvement:

- Before soft-deleting, query `ProductRepository.countByCategoryId(id)`
- If count > 0, throw `ApiException.conflict("Cannot delete category with {n} assigned products. Reassign or move products first.")`
- Optionally support `?forceReassign=targetCategoryId` to auto-move products as part of the delete

---

7.2 `getCategoryTree()` Has No Depth Limit

Current state: `CategoryService.getCategoryTree()` loads `findRootCategories()` with `@EntityGraph(attributePaths = {"subcategories"})`. Spring then eagerly loads all subcategories recursively.

Problem: A deep or wide category tree could load thousands of categories in a single call, causing N+1 queries and memory pressure.

Improvement:

- Add `maxDepth` parameter to `GET /categories/tree?maxDepth=3`
- `CategoryService.getCategoryTree(tenantId, maxDepth)` — prune tree nodes beyond the specified depth before returning
- Default to `maxDepth = 5`; cap at 10

7.3 Hardcoded `null` for `tenantId` in `CategoryController`

Current state: `CategoryController.getTenantIdFromAuthentication()` always returns `null` with a `// implement based on your auth structure` comment.

Problem: Multi-tenancy is modelled on `Category` (the field exists, queries use it) but is never enforced. All tenants' categories are mixed together in every query.

Improvement:

- Integrate with `CurrentUserService` from `user_identity` module to extract `tenantId` from the JWT principal
- Or add a `tenantId` claim to the JWT in `JwtTokenProvider` and read it in `CategoryController`
- All category queries should automatically scope to the authenticated user's tenant — never expose another tenant's categories

---

7.4 Cache Invalidation Is Too Broad

Current state: Every `@CacheEvict` in `CategoryService` and `BrandService` uses `allEntries = true`, wiping the entire cache on every write.

Problem: With many concurrent admins, every category or brand save invalidates the cache for all tenants, causing a cache stampede under load.

Improvement:

- Scope cache keys by `tenantId`: `@CacheEvict(value = "categories", key = "'tree:' + #tenantId")`
- Only evict the specific tenant's cache entries on write
- Use `@CacheEvict(key = "#id")` for single-entity caches

---

8. Improvements — Brands

8.1 `Brand.productCount` Can Drift Out of Sync

Current state: `Brand.productCount` is updated via `BrandService.updateProductCount(id, delta)` — a manual delta operation. If a product is deleted directly through `ProductService` without calling `BrandService.updateProductCount()`, the count becomes stale.

Problem: No coupling exists between `ProductService` and `BrandService`. The count can diverge indefinitely.

Improvement:

- Remove the mutable `productCount` field or make it a computed `@Formula` (Hibernate): `@Formula("(SELECT COUNT(*) FROM products p WHERE p.brand = brand.name AND p.status != 'ARCHIVED')")`
- Or: Add a `ProductEventListener` (`@EntityListener` on `Product`) that calls `BrandService.recalculateProductCount(brandName)` on `@PostPersist` , `@PostUpdate` , `@PostRemove`
- Or: Add a `@Scheduled` nightly job that reconciles all brand product counts from scratch

---

8.2 `BrandService.deleteBrand()` Is a Hard Delete with No Safeguards

Current state: `brandRepository.delete(brand)` — permanent deletion with no check for associated products.

Problem: Deleting a brand that has 500 products leaves those products with a dangling `brand` string and no valid brand reference.

Improvement:

- Check `ProductRepository.countByBrand(brand.getName())` before deletion
- If count > 0, throw `ApiException.conflict("Cannot delete brand with {n} products. Reassign products first or use merge.")`
- Add soft-delete to `Brand` : `isDeleted` , `deletedAt` , `deletedBy` fields (like `Category` already has)

8.3 `BrandController` Does Not Wrap Responses in `ApiResponse`

Current state: `BrandController` returns `ResponseEntity<BrandDto.Response>` directly. `ProductController` and `CategoryController` both wrap in `ApiResponse<T>` .

Problem: Inconsistent API response envelope across the catalog module. The frontend Angular client must handle two different response shapes.

Improvement:

- Wrap all `BrandController` responses in `ApiResponse<T>` to match the rest of the API
- Standardise HTTP status codes: `201 Created` for POST, `200 OK` for GET/PUT/PATCH, `204 No Content` for DELETE

---

9. Improvements — Architecture & Cross-Cutting

9.1 `Product.category` and `Product.brand` Are Plain Strings — Not Foreign Keys

Current state: `Product.category` (String), `Product.categoryId` (String), and `Product.brand` (String) have no database-level foreign key to `Category` or `Brand`.

Problem: You can create a product with `category = "Nonexistent"` and `categoryId = "fake-uuid"` and it saves without error. Referential integrity is entirely absent.

Improvement:

- Replace `Product.category` (String) + `Product.categoryId` (String) with a proper `@ManyToOne Category category` JPA relationship with `@JoinColumn(name = "category_id", nullable = false)`
- Replace `Product.brand` (String) with `@ManyToOne Brand brand` JPA relationship
- Migrate `ProductFilterDTO`, `ProductSpecification`, `ProductMapper`, and `ProductController` to use entity IDs instead of name strings
- Add `@Index` on `category_id` and `brand_id` foreign key columns

9.2 No `@PreAuthorize` on Any Catalog Controller

Current state: `ProductController`, `CategoryController`, and `BrandController` have zero `@PreAuthorize` or `@Secured` annotations. Any authenticated user can create, update, or delete products.

Problem: The `user_identity` module has a full permission system — but the catalog module has no equivalent permission checks. A read-only user can delete products.

Improvement:

- Define catalog permissions in `PermissionRegistry`:
  - `catalog:product:read`, `catalog:product:create`, `catalog:product:update`, `catalog:product:delete`, `catalog:product:publish`, `catalog:product:import`
  - `catalog:category:read`, `catalog:category:create`, `catalog:category:update`, `catalog:category:delete`
  - `catalog:brand:read`, `catalog:brand:create`, `catalog:brand:update`, `catalog:brand:delete`

- Add `@PreAuthorize("hasAuthority('catalog:product:create')")` to all write endpoints
- Add `catalog:*` permissions as a `PermissionCategory.CATALOG_MANAGEMENT` category

9.3 URL Inconsistency Between Controllers

Current state:

- `ProductController` maps to `/products` (no `/api/v1` prefix)
- `BrandController` maps to `/api/v1/brands`
- `CategoryController` maps to `/v1/categories` (no `/api` prefix)

Problem: Three different URL patterns within the same module. The frontend must hardcode three different base paths.

Improvement:

- Standardise all catalog controllers to `/api/v1/catalog/products`, `/api/v1/catalog/categories`, `/api/v1/catalog/brands`
- Or use a global `server.servlet.context-path=/api/v1` in `application.properties` and use `/catalog/products`, `/catalog/categories`, `/catalog/brands` in controllers

---

9.4 No Global Exception Handling for Catalog RuntimeExceptions

Current state: `CategoryService` and `BrandService` throw raw `RuntimeException("Category not found: " + id)`. `ProductService` uses `ApiException`. `GlobalExceptionHandler` exists in common module.

Problem: Raw `RuntimeException` thrown from services produces a 500 Internal Server Error response instead of a proper 404.

Improvement:

- Replace all `throw new RuntimeException(...)` in `CategoryService` and `BrandService` with `throw ApiException.notFound(...)` and `throw ApiException.conflict(...)` — matching the pattern already used in `ProductService`
- Ensure `GlobalExceptionHandler` covers `ApiException` with proper HTTP status mapping

9.5 `@Cacheable` Annotations Without Cache Manager Configuration

Current state: `CategoryService` and `BrandService` use `@Cacheable(value = "categories")` and `@Cacheable(value = "brands")`, but `build.gradle` has the Spring Cache starter commented out and no `CacheManager` bean is configured.

Problem: Spring falls back to `NoOpCacheManager` silently — the cache annotations do nothing, but the code appears to be caching. This is a silent misconfiguration.

Improvement:

- Uncomment `spring-boot-starter-cache` and `caffeine` in `build.gradle`
- Configure a `CaffeineCacheManager` bean with appropriate TTLs (e.g. 5 minutes for category tree, 10 minutes for brand list)
- Or remove the `@Cacheable` annotations entirely until caching is properly configured — misleading no-op annotations are worse than no annotations

---

10. Priority Summary Table

Missing Features

Priority	#	Feature	Module
🔴 Critical	1.10	Product archiving (replace hard delete)	Product
🔴 Critical	1.6	Variant-level price, SKU, stock	Product
🔴 Critical	1.7	Product import (CSV/Excel)	Product
🔴 Critical	5.1	Catalog audit log (who changed what)	All
🔴 Critical	2.1	Category → product assignment & count	Category
🟠 High	1.1	Product cloning / duplication	Product
🟠 High	1.2	Publishing workflow & state machine	Product
🟠 High	1.3	Product scheduling (publish / unpublish date)	Product
🟠 High	1.5	Multi-image management endpoints	Product

🟠 High	1.8	Product export (CSV/Excel)	Product
🟠 High	1.12	Bulk price update across category/brand	Product
🟠 High	2.2	Category attribute templates	Category
🟠 High	5.2	Catalog health dashboard	All
🟡 Medium	1.4	Product versioning / change history	Product
🟡 Medium	1.9	Related / cross-sell / up-sell products	Product
🟡 Medium	2.3	Bulk product move between categories	Category
🟡 Medium	3.1	Brand product listing	Brand
🟡 Medium	3.2	Brand analytics	Brand
🟡 Medium	4.1	Search synonym management	Search
🟡 Medium	5.3	SEO audit endpoint	All
🟢 Lower	1.11	Product comparison endpoint	Product
🟢 Lower	2.4	Category-level pricing rules	Category
🟢 Lower	3.3	Brand merge / consolidation	Brand
🟢 Lower	3.4	Brand SEO fields	Brand
🟢 Lower	4.2	Product ranking / boost rules	Search
🟢 Lower	4.3	Faceted search configuration	Search

Improvements

Priority	#	Improvement	Module
🔴 Critical	9.2	Add `@PreAuthorize` permission checks to all controllers	Architecture
🔴 Critical	9.1	Replace string category/brand with FK relationships	Architecture
🔴 Critical	9.4	Replace raw `RuntimeException` with `ApiException`	Architecture

🟠 High	6.1	Replace string status/visibility with enums	Product
🟠 High	6.3	Remove unbounded `getAllProducts()`	Product
🟠 High	6.7	Block base64 image storage in DB	Product
🟠 High	7.1	Check for products before deleting a category	Category
🟠 High	8.2	Add safeguard to `BrandService.deleteBrand()`	Brand
🟠 High	9.5	Fix no-op cache configuration	Architecture
🟡 Medium	6.2	Fix bulk ops to return per-item results	Product
🟡 Medium	6.4	Fix full-table LIKE scan in `search()`	Product
🟡 Medium	6.5	Replace `Map<String, Object>` patch with typed DTO	Product
🟡 Medium	6.6	Fix `deleteProduct()` cascade & cleanup	Product
🟡 Medium	7.3	Implement `tenantId` extraction from JWT	Category
🟡 Medium	8.1	Fix `productCount` drift on `Brand`	Brand
🟡 Medium	8.3	Wrap `BrandController` responses in `ApiResponse`	Brand
🟢 Lower	7.2	Add `maxDepth` param to `getCategoryTree()`	Category
🟢 Lower	7.4	Scope cache eviction by tenantId	Category/Brand
🟢 Lower	9.3	Standardise URL patterns across all controllers	Architecture

---

Generated for Commerce Pro · catalog module · `commerce-pro-application-main`